

Programmation Graphique C++

ATI M1



Opérations binaires



Opération « ET »

Si l'un des bits vaut 0
alors le résultat vaut
toujours 0

$$0 \text{ ET } 0 = 0$$

$$0 \text{ ET } 1 = 0$$

$$1 \text{ ET } 0 = 0$$

$$1 \text{ ET } 1 = 1$$

$$A \text{ ET } B$$

$$A . B$$



Opération « OU »

Si l'un des bits vaut 1
alors le résultat vaut
toujours 1

$$0 \text{ OU } 0 = 0$$

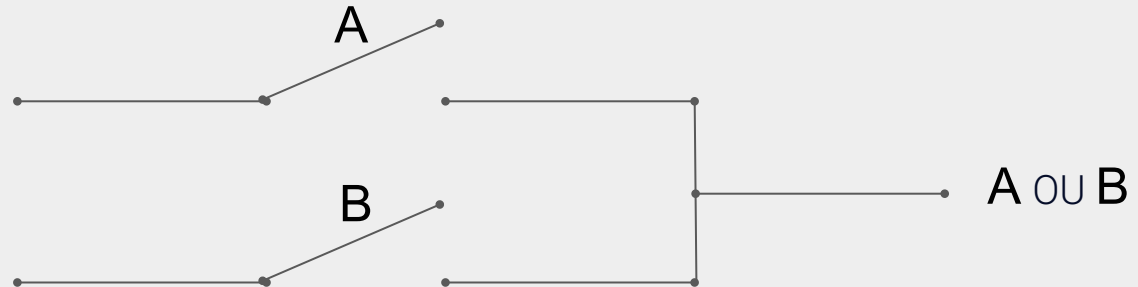
$$0 \text{ OU } 1 = 1$$

$$1 \text{ OU } 0 = 1$$

$$1 \text{ OU } 1 = 1$$

$$A \text{ OU } B$$

$$A + B$$



Opération « NON »

L'opération « NON »
inverse tous les bits

$$\overline{0} = 1$$

$$\overline{1} = 0$$

$$\overline{0000} = 1111$$

$$\overline{1111} = 0000$$

$$\overline{0101} = 1010$$

$$\overline{1100} = 0011$$



Opération « XOR » (ou « OU exclusif »)

Si un seul des bits vaut 1
alors le résultat vaut 1 ;
sinon, il vaut 0

ou

si tous les bits sont égaux, le
résultat vaut 0
si les bits diffèrent, le résultat
vaut 1

$$0 \text{ XOR } 0 = 0$$

$$0 \text{ XOR } 1 = 1$$

$$1 \text{ XOR } 0 = 1$$

$$1 \text{ XOR } 1 = 0$$

$$A \text{ XOR } B$$

$$A \oplus B$$

$$0000 \text{ XOR } 0000 = 0000$$

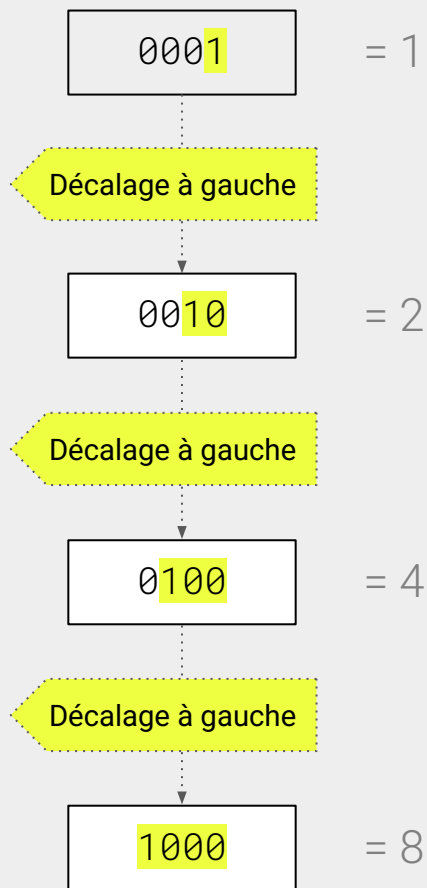
$$1100 \text{ XOR } 0011 = 1111$$

$$1111 \text{ XOR } 1111 = 0000$$

$$1010 \text{ XOR } 1111 = 0101$$



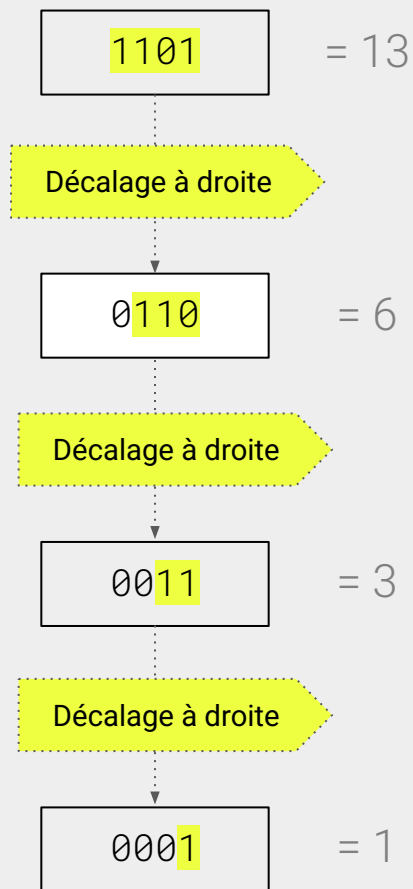
Décalage à gauche



...revient à multiplier par 2

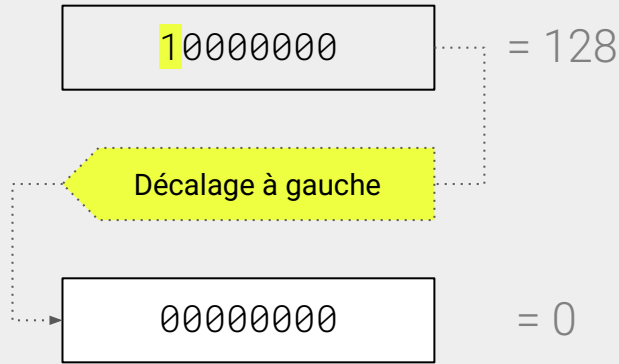


Décalage à droite

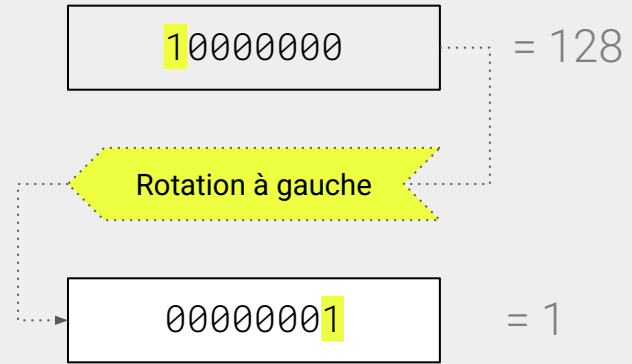


...revient à diviser par 2

Décalage ou rotation



OU

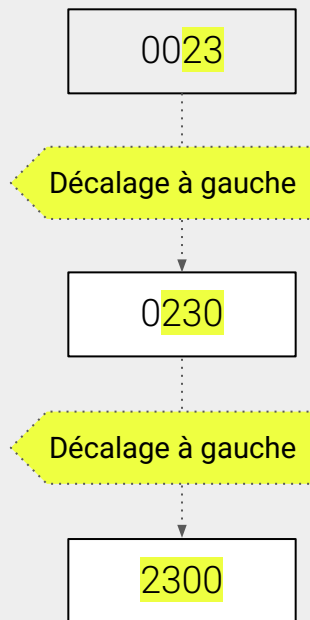


Les bits en excès sont supprimés

Les bits en excès sont réinjectés



Et cela fonctionne aussi en décimal...



Bit de signe

Dans les nombres binaires signés, **un bit** est souvent réservé pour indiquer le signe du nombre.

Il s'agit généralement du bit le plus à gauche (aussi appelé bit de poids fort ou MSB).

- **0** à cette position indique un nombre **positif**.
- **1** indique un nombre **négatif**.

Les nombres binaires négatifs peuvent être représentés de plusieurs manières :

- signe et valeur absolue
- complément à un
- complément à deux

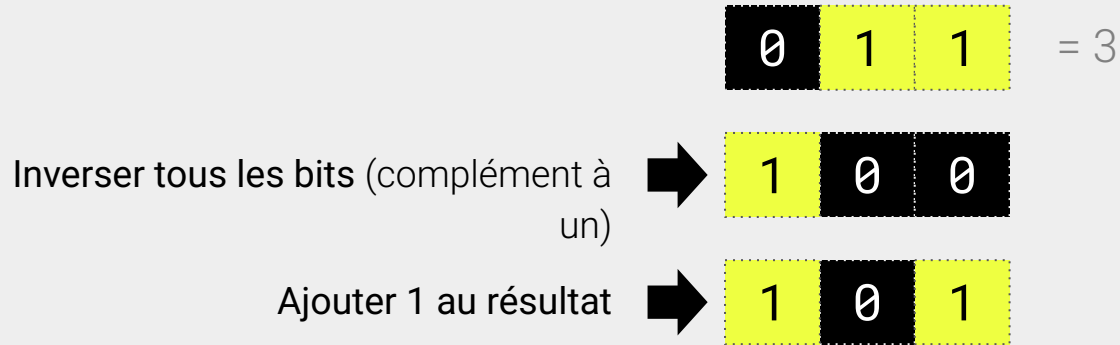
Le complément à deux est la méthode **la plus courante**.



Complément à deux

Le complément à deux est couramment utilisé car il simplifie le matériel nécessaire à l'addition et à la soustraction.

Pour calculer le complément à deux d'un nombre :



(et le complément à deux supprime l'ambiguïté entre +0 et -0 en binaire)



Le complément à deux, ou « pourquoi $-3 + 3$ vaut zéro » ?

La somme d'un nombre et de son complément à deux est toujours nulle.

Par exemple, en représentation sur **8 bits** en complément à deux :

$$\begin{array}{r} \begin{array}{cccccc|cc} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{array} & = 3 \\ + & \begin{array}{cccccc|cc} 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \end{array} & = -3 \\ \hline = & \begin{array}{c} 1 \end{array} \begin{array}{cccccc|cc} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} & = 0 \end{array}$$

Le résultat est **10000000** mais comme nous n'avons que **8 bits**, le bit le plus à gauche **déborde**, ce qui laisse **00000000** égal à **zéro**.



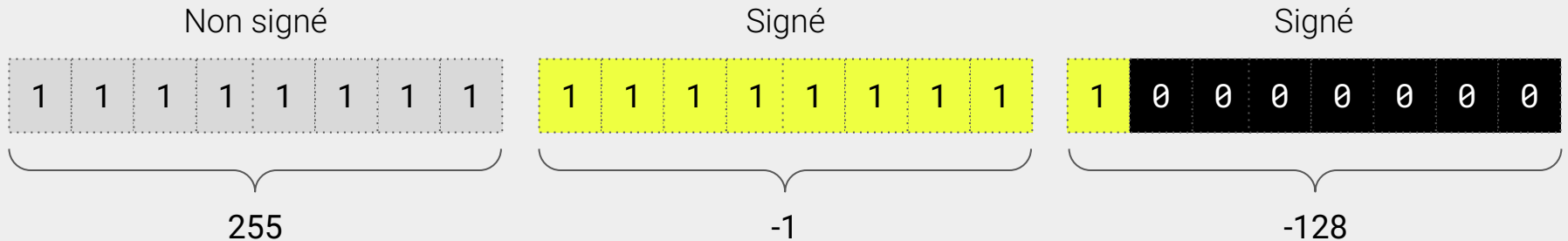
Non signé ou signé

Non signé ou signé

- **Non signé** : ne représente que des nombres positifs, mais offre une plage de valeurs positives plus étendue.
- **signé** : représente les nombres positifs et négatifs, mais offre une plage de valeurs positives plus restreinte.

Dans un système 8 bits :

- Un octet non signé représente les nombres de 0 à 255 ($2^8 - 1$).
- Un octet signé en complément à deux représente les nombres de -128 à 127.



Non signé

11111111	= 255
11111000	= 248
11000000	= 192
10000000	= 128
01111111	= 127
01000000	= 64
00001000	= 8
00000001	= 1

Signé

01111111	= 127
01000000	= 64
00001000	= 8
00000001	= 1
11111111	= -1
11111000	= -8
11000000	= -64
10000000	= -128



Une conséquence sur le stockage des données audio



Prélude en do de Bach (16 bits, signé)



Prélude en do de Bach (8 bits, non signé)

Que se passe-t-il maintenant si vous lisez vos données non signées *comme si elles étaient signées* ?

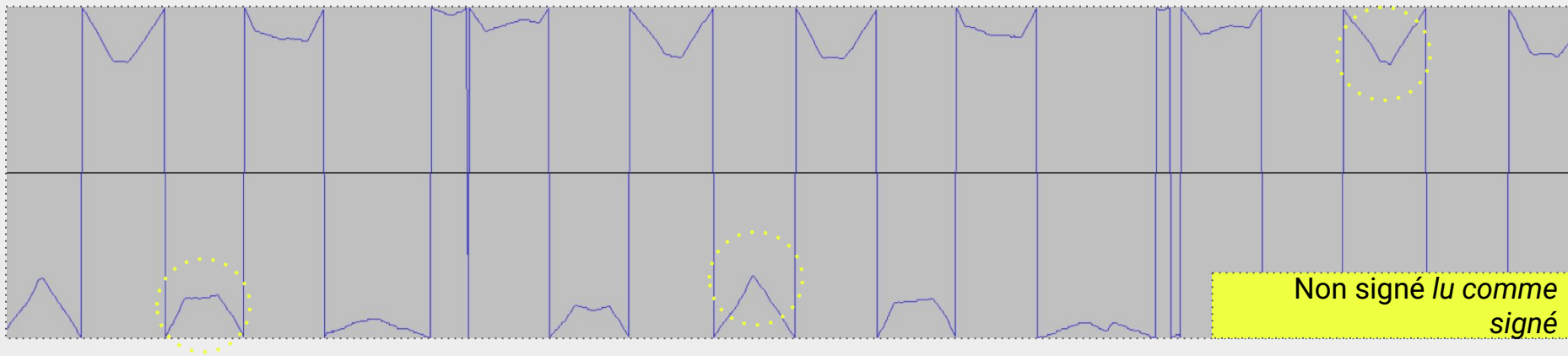
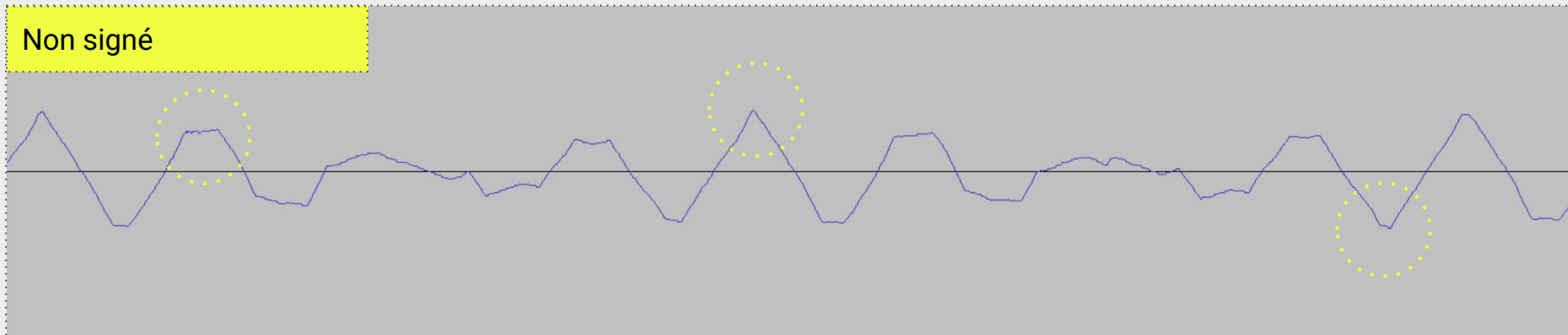


Prélude en do de Bach (8 bits, non signé, *lu comme signé*)

ne cliquez pas ici

Une conséquence sur le stockage des données audio

Non signé



Non signé lu comme
signé

Opérations bit à bit



Compter en binaire, décimal et hexadécimal

Binaire

0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------

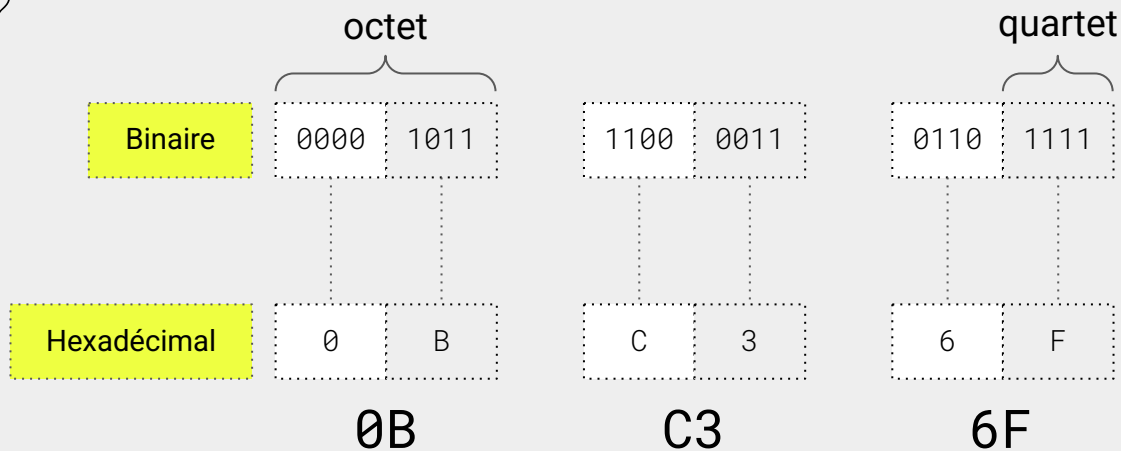
Décimal

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

Hexadécimal

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pourquoi le binaire et l'hexadécimal fonctionnent-ils bien ensemble ?

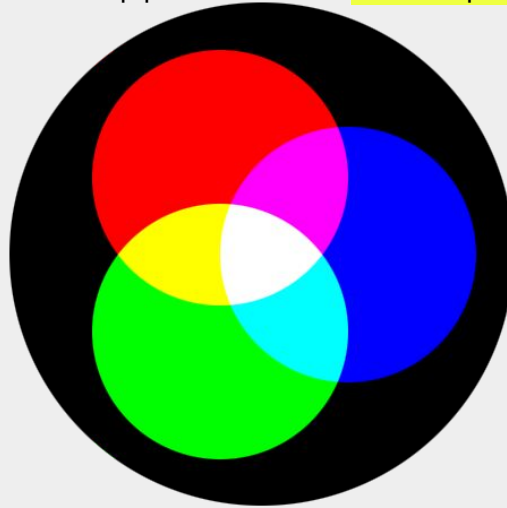


L'hexadécimal offre une lecture plus claire du contenu d'un octet,
il est plus facile de lire **2 quartets** que **8 bits distincts**.



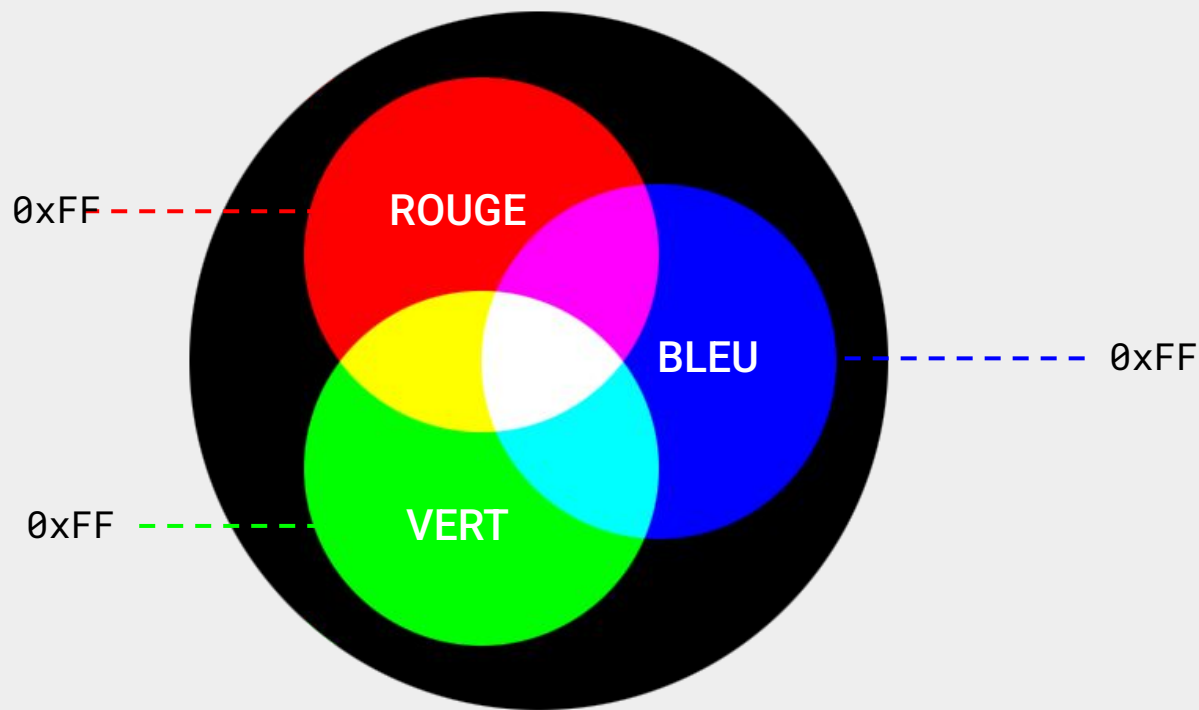
Conversion de l'espace colorimétrique par synthèse additive

Dans un ordinateur, les couleurs sont représentées
en RVB
R, V et B sont appelées des « composantes »



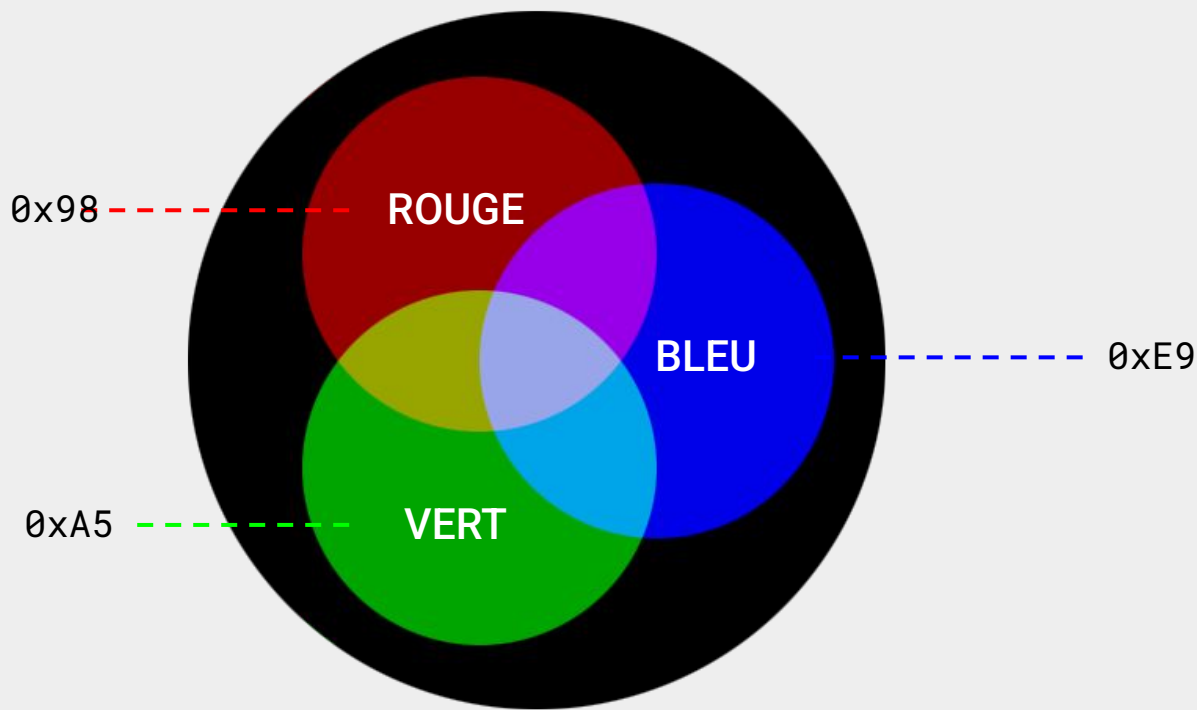
RVB = ROUGE VERT BLEU

Conversion de l'espace colorimétrique par synthèse additive



`0xFF0000 | 0x00FF00 | 0x00FF00 = 0xFFFF00`

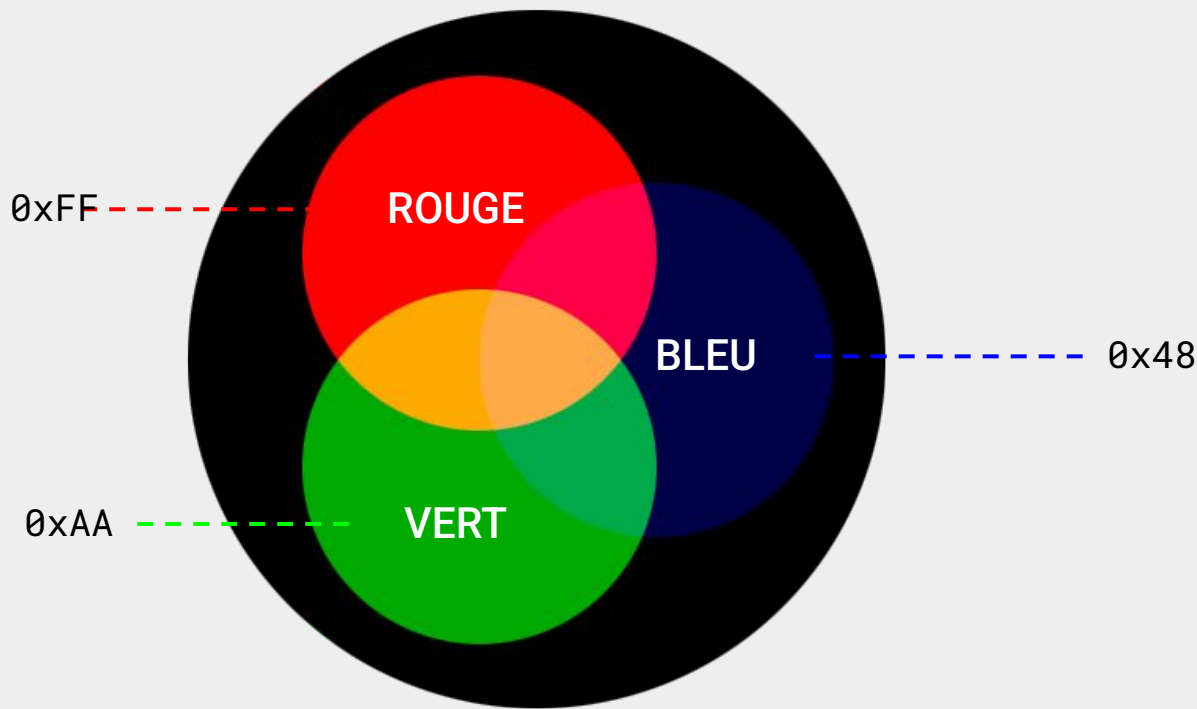
Conversion de l'espace colorimétrique par synthèse additive



0x980000 | 0x00A500 | 0x00FFE9 = 0x98A5E9



Conversion de l'espace colorimétrique par synthèse additive



0xFF0000 | 0x00AA00 | 0x00FF48 = 0xFFAA48



Conversion de l'espace colorimétrique par synthèse additive

quelques exemples supplémentaires...

0x00FF00	 RGB(0, 255, 0)			RGB(255, 0, 0)	0xFF0000
0xFFFFFFFF	RGB(255, 255, 255)			RGB(0, 255, 255)	0x00FFFF
0x808080	RGB(128, 128, 128)			RGB(0, 0, 255)	0x0000FF
0xFFFF00	RGB(255, 255, 0)			RGB(0, 0, 0)	0x000000
0xFF8040	RGB(255, 128, 64)			RGB(255, 0, 255)	0xFF00FF

Conversion de l'espace colorimétrique par opérations bit à bit

0x00FF00

0b 

0xFFFF10

0b 

0xFFAA48

0b 

0xEDA27B

0b 

Conversion de l'espace colorimétrique par opérations bit à bit

11110000 >> 1

0b_____

10100101 >> 2

0b_____

10101010 >> 3

0b_____

00001111 << 4

0b_____



Conversion de l'espace colorimétrique par opérations bit à bit

1111000011110000 >> 4

0b_____

1010010110101010 >> 8

0b_____

1010000011111010 >> 12

0b_____

0000111110101010 << 8

0b_____



Conversion de l'espace colorimétrique par opérations bit à bit

RVB 24 (8 BITS par composante) :

- VERT RVB(0, 255, 0) 0x00FF00
- BLANC RVB(255, 255, 255) 0xFFFFFFFF
- GRIS RVB(127, 127, 127) 0x7F7F7F
- JAUNE RVB(255, 255, 0) 0xFFFF00
- ROSE RVB(227, 162, 120) 0xEDA27B



RVB 12 (4 BITS par composante) :

- ROSE RVB(224, 160, 112) 0x0EA7



Conversion de l'espace colorimétrique par opérations bit à bit

Nous voulons réduire la largeur en bits de chaque composante, de 8 bits (un octet) à 4 bits (un quartet) par composante.

RVB 24 (8 BITS par composante)

0xEDA27B



RVB 12 (4 BITS par composante)

0xEA7



Algorithme en pseudo-code

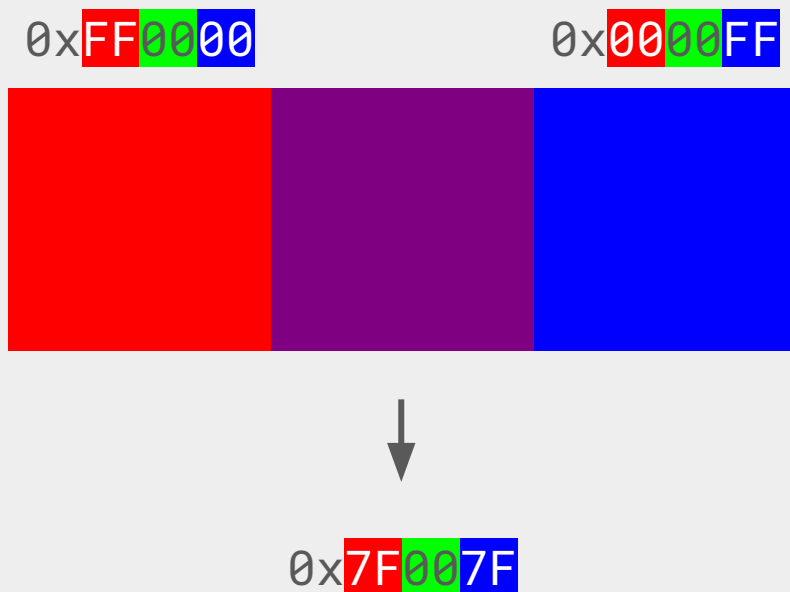
```
function RGB24_TO_RGB12(rgb24)
{
    var r, g, b // Stockage temporaire de chaque composante couleur

    r = (rgb24 AND 0xF00000) >> 12 // MASQUER le rouge puis le DÉCALER
    g = (rgb24 AND 0x00F000) >> 8  // MASQUER le vert puis le DÉCALER
    b = (rgb24 AND 0x00F0)  >> 4  // MASQUER le bleu puis le DÉCALER
    // Remarque : le « masquage » consiste à utiliser un opérateur AND

    return (r OR g OR b) // Réinjecter chaque composante avec OR
}
```



Conversion de l'espace colorimétrique par opérations bit à bit



Algorithme en pseudo-code

couleur1

0x**FF****00****00**

couleur2

0x**00****00****FF**

- extraire la composante ROUGE pour la ramener de 0x**FF**FFFF to 0x0000**FF** (= 0xFF)
- extraire (et masquer ?) le VERT pour le ramener de 0xFF**FF**FF to 0x0000**FF** (= 0xFF)
- extraire le BLEU pour le ramener de 0xFFFF**FF** vers 0x0000**FF** (= 0xFF)
- répéter cette opération pour les DEUX couleurs (couleur1 et couleur2)
- calculer séparément la MOYENNE de chaque composante des deux couleurs
- recombinaison chaque moyenne ROUGE, VERTE et BLEUE en une seule valeur RVB :
0x**FF****FF****FF**

Résultat

0x**7F****00****7F**



Algorithme en pseudo-code

```
function MIX_RGB24(color_a, color_b)
{
    var mix, Ra, Ga, Ba, Rm, Gm, Bm // Stockage temporaire de chaque composante couleur
    Ra = (color_a AND 0xFF0000) >> 16 // MASQUER le rouge puis le DÉCALER
    Ga = (color_a AND 0x00FF00) >> 8 // MASQUER le vert puis le DÉCALER
    Ba = (color_a AND 0x00FF) // MASQUER le bleu
    Rb = (color_b AND 0xFF0000) >> 16 // MASQUER le rouge puis le DÉCALER
    Gb = (color_b AND 0x00FF00) >> 8 // MASQUER le vert puis le DÉCALER
    Bb = (color_b AND 0x00FF) // MASQUER le bleu
    Rm = ((Ra + Rb) >> 1) << 16 // Diviser par 2 puis replacer à la position du ROUGE
    Gm = ((Ga + Gb) >> 1) << 8 // Diviser par 2 puis replacer à la position du VERT
    Bm = (Ba + Bb) >> 1 // Diviser par 2 (déjà à la position du BLEU)
    mix = Rm OR Gm OR Bm // Recombiner les composantes ROUGE, VERTE et BLEUE
    return mix
}
```

